

# Visión por computador para un jugador físico de Onitama (cámara fija)

---

Guía de enfoque, ideas y enlaces (documentación, repositorios y papers)

## 1. Objetivo del módulo de visión

Dado un portátil con cámara apuntando al tablero y a las cartas, el objetivo es estimar el estado del juego en cada turno:

- Piezas: 2 tipos (Maestro/Alumno)  $\times$  2 colores (jugador A/jugador B).
- Posición: tablero 5 $\times$ 5 (casillas discretas).
- Cartas: 2 cartas activas del jugador que mueve + 1 carta “por venir” a un lado (según montaje).
- Salida: estado discreto (matriz 5 $\times$ 5 + IDs de cartas) listo para el motor (minimax con poda alfa-beta).

## 2. Flujo de trabajo recomendado (robusto para cámara fija)

### 2.1. Fijar el sistema de coordenadas del tablero (homografía)

La pieza clave para robustez es convertir cada frame a una vista cenital normalizada del tablero. Con una homografía, cualquier punto de la imagen se mapea a coordenadas del tablero; después, cada casilla del 5 $\times$ 5 queda en una región fija.

Opciones prácticas:

- Homografía a partir de 4 puntos (esquinas del tablero) y ‘warpPerspective’.
- Aún más estable: colocar 4 marcadores ArUco/ChArUco en el marco del tablero (o en una base impresa) para estimar la pose/homografía de forma consistente pese a cambios de iluminación u oclusiones parciales.

### 2.2. Detección/clasificación de piezas (4 clases)

Una vez el tablero está ‘warpeado’ a cenital (por ejemplo 500 $\times$ 500 px), hay tres enfoques típicos:

- YOLO (detección) con 4 clases: maestro\_A, alumno\_A, maestro\_B, alumno\_B. Luego se asigna cada detección a casilla usando el centro del bounding box.

- YOLO (segmentación) si hay solapes o bordes difíciles: máscara por objeto suele ayudar a localizar mejor el centro.
- Baseline clásico (sin deep learning): segmentación por color + forma sobre la imagen cenital. Puede servir como comparación, pero es más sensible a sombras/reflejos.

### 2.3. Reconocimiento de cartas (2 activas + 1 lateral)

Las cartas pueden resolverse de dos maneras muy habituales:

- Detección de 'carta' (YOLO) + clasificación del tipo de carta (16 clases) en un clasificador (CNN / YOLO classify).
- Recorte por ROI (si el montaje es estable) + normalización de perspectiva (homografía) + comparación con plantillas (ORB/KAZE + matching, o hash perceptual).

### 2.4. Estabilización temporal (antirruido)

Como el juego es por turnos, conviene aplicar un filtro temporal para evitar falsos positivos por manos, motion blur, etc.

- Aceptar un estado solo si se repite N frames consecutivos (o por mayoría en una ventana).
- Opcional: detectar 'mano presente' (clase adicional) y congelar inferencia mientras la mano está en escena.

## 3. MVP sugerido para el TFG

Un camino mínimo viable, fuerte y defendible en memoria del TFG:

1. Marcadores ArUco/ChArUco + homografía + warp a vista cenital del tablero.
2. YOLO detección (4 clases) sobre imagen cenital para piezas.
3. Cartas: recorte por ROI fijo (o detección YOLO) + clasificador 16 clases.
4. Debounce/estabilización temporal para confirmar estados.
5. Conversión a estado discreto → minimax alfa-beta.

## 4. Enlaces recomendados (URLs en texto plano)

Nota: todos los enlaces se incluyen como texto plano (no se insertan hipervínculos en el documento).

### 4.1. OpenCV: homografía, alineamiento y transformaciones

- [https://docs.opencv.org/4.x/d9/dab/tutorial\\_homography.html](https://docs.opencv.org/4.x/d9/dab/tutorial_homography.html)
- [https://docs.opencv.org/4.x/da/d54/group\\_\\_imgproc\\_\\_transform.html](https://docs.opencv.org/4.x/da/d54/group__imgproc__transform.html)
- [https://docs.opencv.org/4.x/d1/de0/tutorial\\_py\\_feature\\_homography.html](https://docs.opencv.org/4.x/d1/de0/tutorial_py_feature_homography.html)
- [https://docs.opencv.org/3.4/d1/de0/tutorial\\_py\\_feature\\_homography.html](https://docs.opencv.org/3.4/d1/de0/tutorial_py_feature_homography.html)
- <https://learnopencv.com/homography-examples-using-opencv-python-c/>
- <https://learnopencv.com/image-alignment-feature-based-using-opencv-c-python/>

#### 4.2. OpenCV: ArUco / ChArUco (detección, calibración, pose)

- [https://docs.opencv.org/3.4/d9/d6d/tutorial\\_table\\_of\\_content\\_aruco.html](https://docs.opencv.org/3.4/d9/d6d/tutorial_table_of_content_aruco.html)
- [https://docs.opencv.org/3.4/d5/dae/tutorial\\_aruco\\_detection.html](https://docs.opencv.org/3.4/d5/dae/tutorial_aruco_detection.html)
- [https://docs.opencv.org/4.x/d1/dcb/tutorial\\_aruco\\_faq.html](https://docs.opencv.org/4.x/d1/dcb/tutorial_aruco_faq.html)
- <https://learnopencv.com/augmented-reality-using-aruco-markers-in-opencv-c-python/>
- <https://pyimagesearch.com/2020/12/21/detecting-aruco-markers-with-opencv-and-python/>

#### 4.3. Ultralytics YOLO (docs oficiales)

- <https://docs.ultralytics.com/>
- <https://docs.ultralytics.com/models/yolov8/>
- <https://docs.ultralytics.com/modes/train/>
- <https://docs.ultralytics.com/tasks/segment/>
- <https://docs.ultralytics.com/tasks/classify/>
- <https://docs.ultralytics.com/usage/cfg/>
- <https://www.ultralytics.com/blog/using-a-vision-ai-model-to-recognize-playing-cards>

#### 4.4. Entrenar YOLO con dataset propio (guías prácticas)

- <https://blog.roboflow.com/how-to-train-yolov8-on-a-custom-dataset/>
- <https://learnopencv.com/train-yolov8-on-custom-dataset/>
- <https://roboflow.com/integration/yolov8>

#### 4.5. Repositorios similares: ajedrez (tablero/piezas → estado)

- <https://github.com/georg-wolflein/chesscog>
- <https://github.com/tmasouris/end-to-end-chess-recognition>
- <https://github.com/andrefakhoury/chess-board-recognition>
- [https://github.com/rondinellimorais/chess\\_recognition](https://github.com/rondinellimorais/chess_recognition)
- [https://github.com/simarmehta/chessAutomation\\_CV](https://github.com/simarmehta/chessAutomation_CV)
- <https://github.com/siromermer/Dynamic-Chess-Board-Piece-Extraction>
- <https://github.com/AI-bootcamp/capstone-project-team01>
- <https://github.com/Dilanya/Chess-Pieces-Detection>
- <https://github.com/shainisan/chess-yolov8>
- <https://github.com/ChapponE/An-Object-Detection-Ramble-with-YOLOv8n-Application-to-Chess-Pieces>

#### 4.6. Repositorios similares: cartas (detección/identificación)

- <https://github.com/EdjeElectronics/OpenCV-Playing-Card-Detector>
- <https://github.com/dofire/OpenCV-Playing-Card-Detection>
- <https://github.com/naderchhab/card-detector>
- [https://github.com/Stormynova/Uno\\_Card\\_detection](https://github.com/Stormynova/Uno_Card_detection)
- <https://github.com/PD-Mera/Playing-Cards-Detection>

- <https://github.com/noorkhokhar99/Playing-Cards-Detection-with-YoloV8>

#### 4.7. Repositorios/pipelines en otros juegos de tablero (inspiración)

- <https://github.com/christiancorro/BackgammonCV>
- <https://medium.com/@nitzankolatacz/how-i-used-machine-learning-to-beat-my-friends-at-backgammon-fb541ec1c0e5>

#### 4.8. Papers y datasets (muy transferibles a Onitama)

- <https://www.mdpi.com/2313-433X/7/6/94>
- <https://arxiv.org/abs/2104.14963>
- <https://vision.fe.uni-lj.si/cvww2016/proceedings/papers/21.pdf>
- <https://arxiv.org/html/2310.04086v3>

#### 4.9. Onitama: reglas, representación de cartas y repos de IA

- <https://www.arcanewonders.com/wp-content/uploads/2021/05/Onitama-Rulebook.pdf>
- <https://cdn.1j1ju.com/medias/8f/3d/2b-onitama-rulebook.pdf>
- <https://github.com/sarah-is-coding/Onitama-AI>
- <https://github.com/arijit-dasgupta/OnitamaAI>
- <https://github.com/TrippW/onitama>
- <https://github.com/maxbenedich/onitama>
- <https://github.com/kezabelle/onitama>
- <https://boardgamegeek.com/thread/1572228/card-movement-and-orientation>
- <https://salvi.chaosnet.org/snippet.html>
- <https://boardgamegeek.com/files/thing/160477>
- <https://boardgamegeek.com/filepage/291555/onitama-base-game-minimal-pnp-and-travel-variant-m>
- <https://boardgamegeek.com/filepage/249135/onitama-cards-2x2-plus-backs-a4>